WK 03: SQL 2 - Databases

Understanding Database Design, Connecting Tools

PPOL 5206 | Massive Data Fundamentals | Spring 2026

WEEK 2 FOUNDATIONS

- Data anomalies
  - why structure matters
- Primary keys & foreign keys
- The "Big Six" SQL clauses
- Querying existing databases
- Lab: SQL Murder Mystery
  - (exploration mode)

WEEK 3 GOALS

- Visual database modeling
  - Entity Relationship Diagrams (ERDs)
- Normalization theory
  - Organizing relational databases
- Translating between diagrams and code
- Database Organization
  - Schemas
- JOINs
  - Recombining normalized data
- CTEs (Again)
  - Elegant query solutions
- Connecting SQL to Python

WEEK 2 vs. WEEK 3

Week 2: Exploration Mode

- Discovered schema by trial
- Many queries to find clues
- "What tables exist?“

Week 3: Mastery Mode

- Know the structure upfront
- Elegant CTEs build logically
- "How do they connect?“

Same mystery. Better tools. Cleaner solution.

ENTITY-RELATIONSHIP DIAGRAMS

Visualizing Database Structure

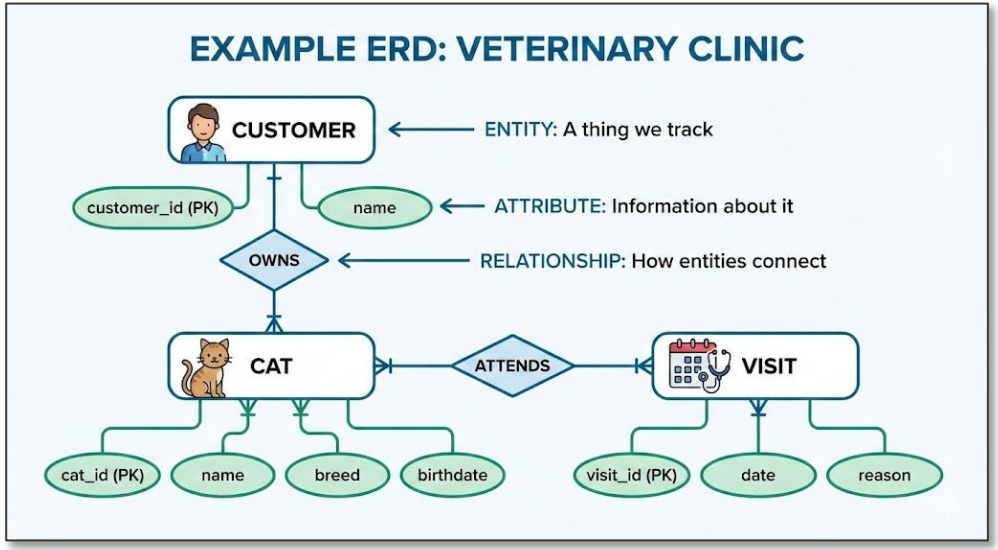
WHY VISUALIZE FIRST?

- Database designers draw before they code
- Communicate structure to non-technical stakeholders
- Reveal relationships not obvious in raw tables
- Serve as documentation for future maintainers
- Required for many government data systems
- Show schema organization, not just tables

"If you can't diagram it, you don't understand it."

THE BUILDING BLOCKS

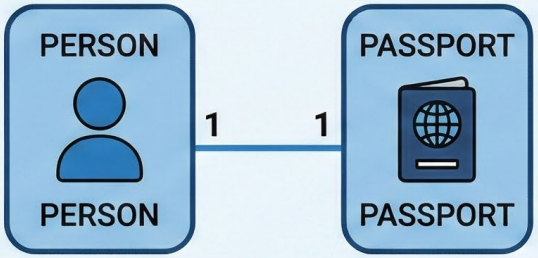
- ENTITY: A thing we track (e.g., Customers, Cats, Visits)
  - Shown as Rectangle
- Attribute: Information about it (e.g., name, email, date)
  - Listed inside Rectangle
- Relationship: How entities connect
  - Lines connecting Rectangles



CARDINALITY: The "how many" of relationships

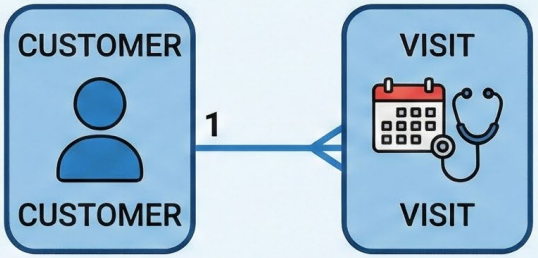
ONE-TO-ONE (1:1)

One person has one passport



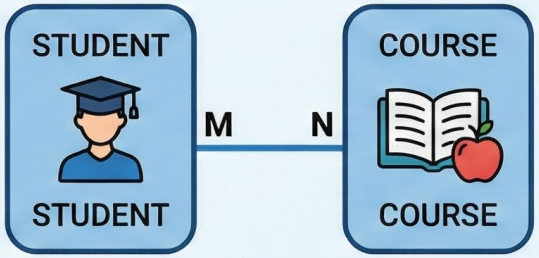
ONE-TO-MANY (1:N)

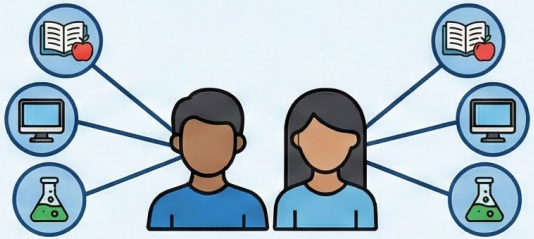
One customer has many visits



MANY-TO-MANY (M:N)

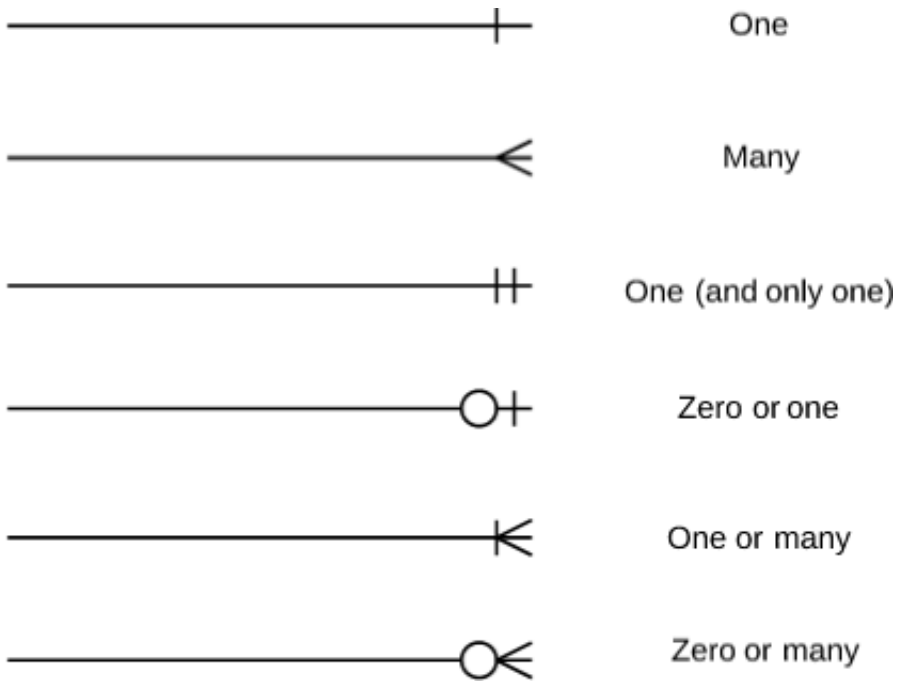
Students enroll in many courses





CROW'S FOOT NOTATION

- Data Table Relationship Types
- Industry standard symbols for cardinality



Source: Entity-Relationship Diagram6 Symbols and Notation | Lucidchart

PRACTICE READING

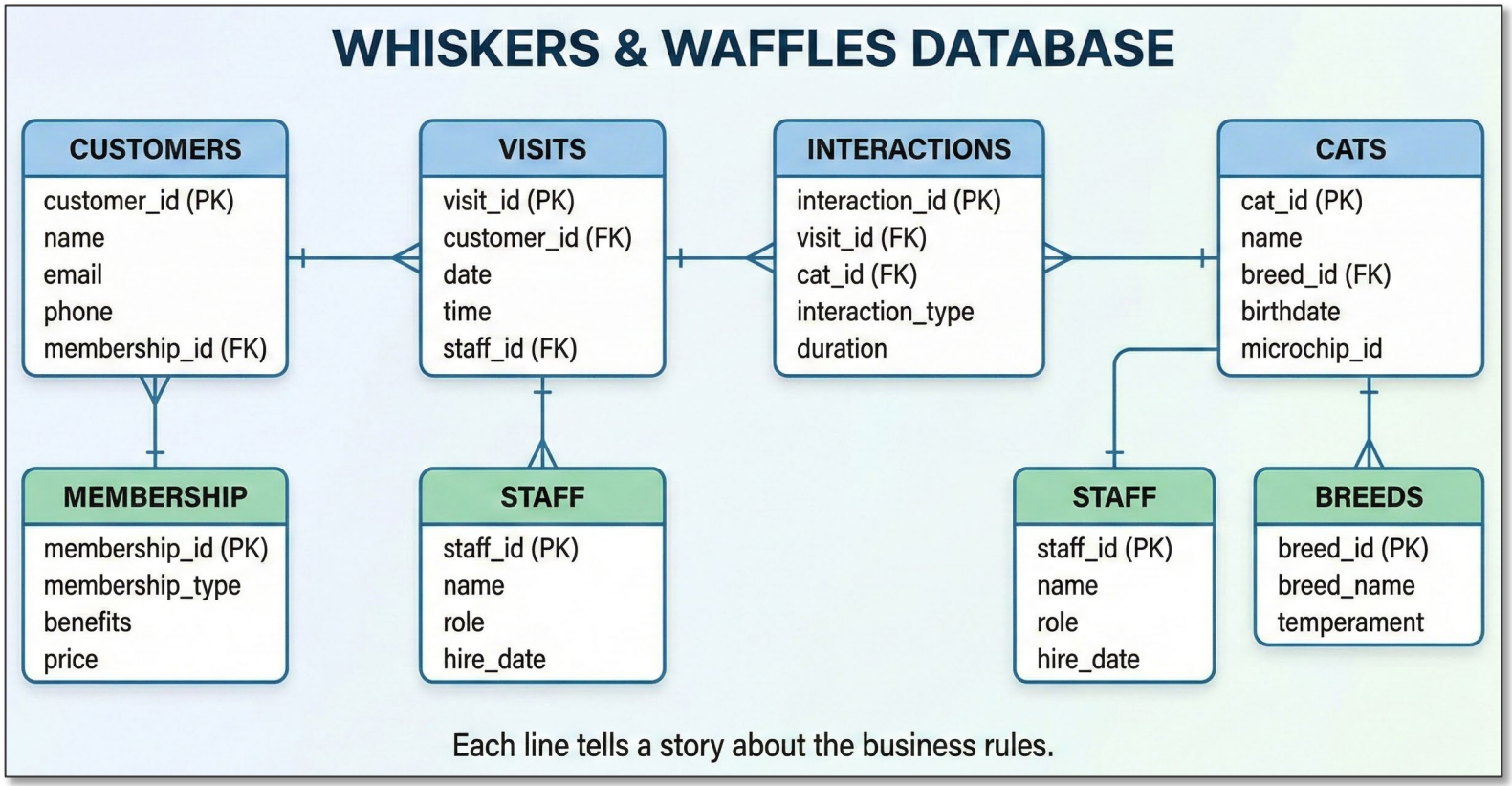
Read left to right:

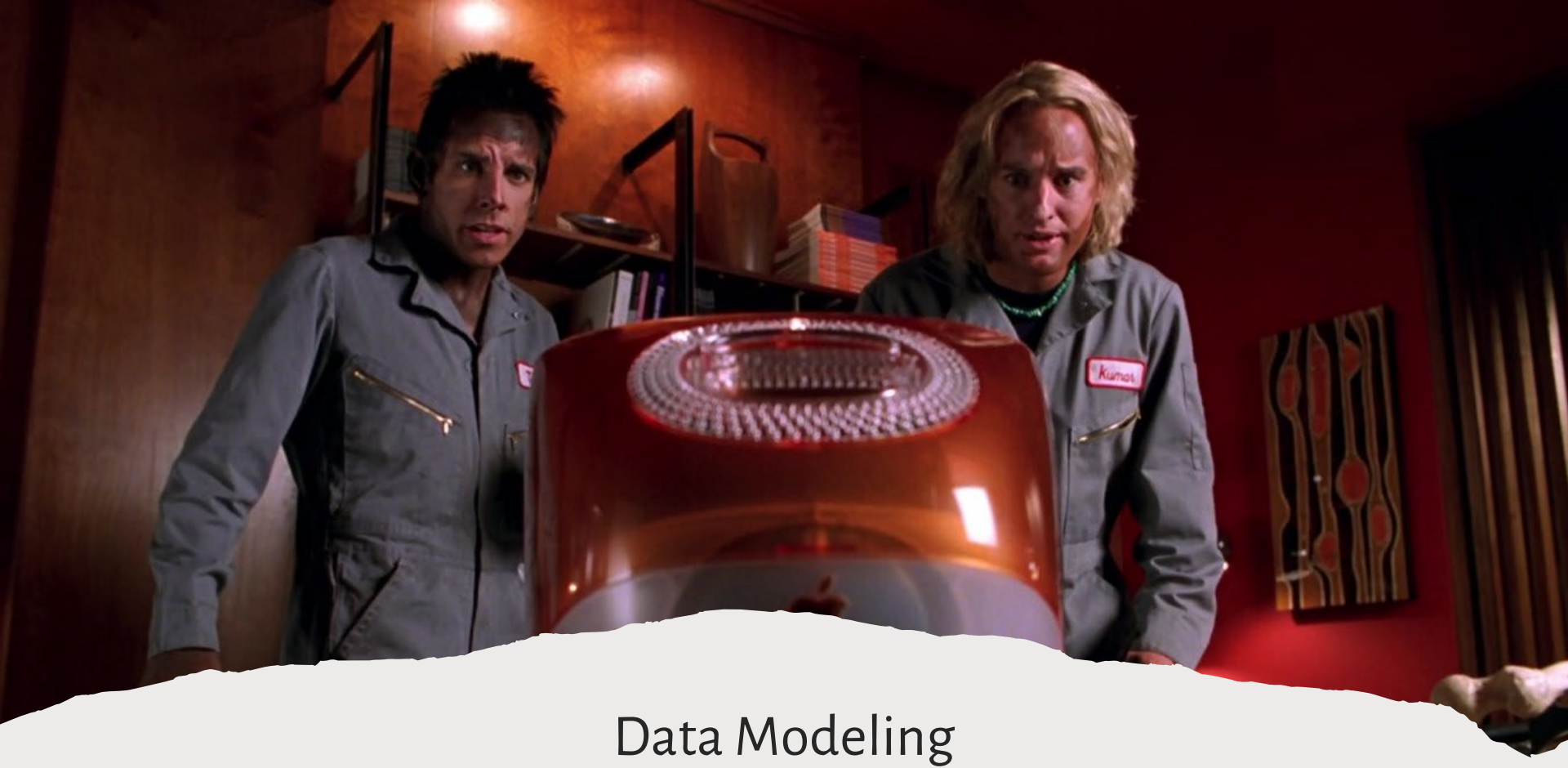
- One cat can have many interactions
- Each interaction involves one cat

Read right to left:

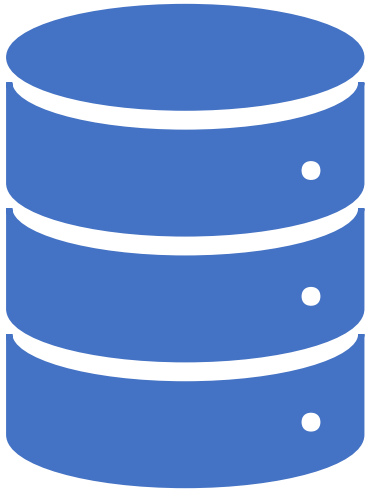
- One visit can have many interactions
- Each interaction belongs to one visit

The Cat Café ERD



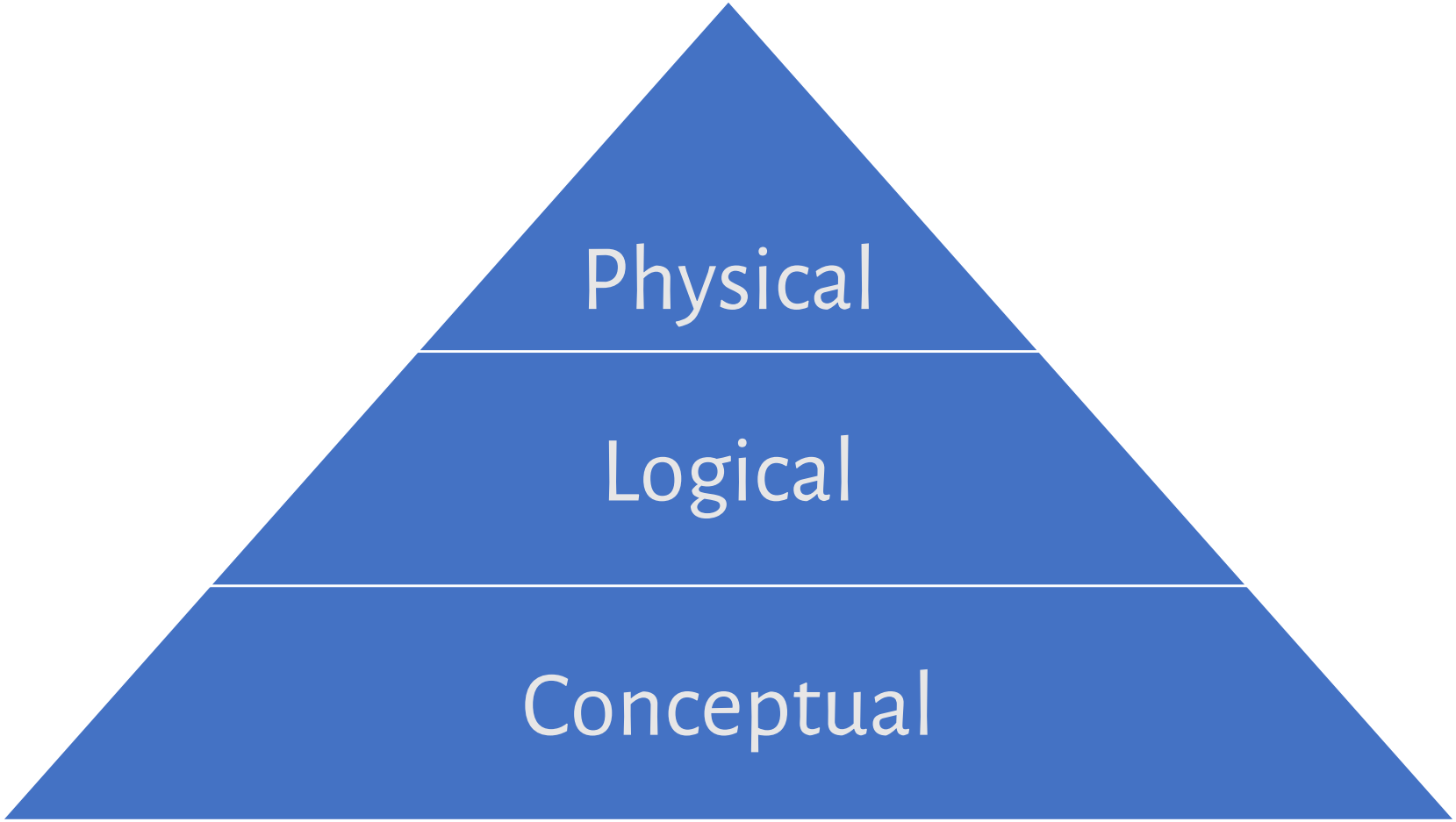


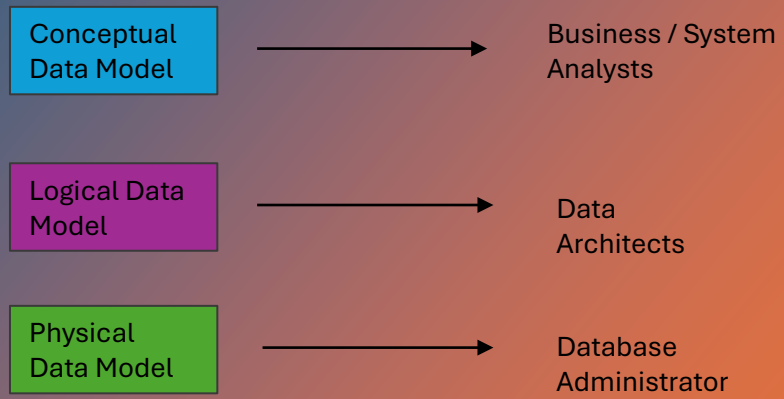
Data Modeling



Introduction to Data Modeling

- **Definition:** Data modeling is the process of documenting a specific, often visual, representation of a system's data elements and their relationships.
- **Purpose:** It helps with organizing data, ensuring consistency, and making data management more efficient.
- There are 3 different types of data models produced in the process of creating the actual database(s) for your systems:
  - Conceptual
  - Logical
  - Physical


Physical
Logical
Conceptual



Why do we need data models?

CONCEPTUAL (Business view):

- What entities exist?
- How do they relate?
- No technical details.

LOGICAL (Designer view):

- Add attributes, keys, data types
- Database-agnostic "blueprint."

PHYSICAL (Developer view):

- Specific to PostgreSQL/MySQL
- Includes indexes, constraints

TOOLS FOR THIS COURSE

VISUAL PARADIGM (Forward engineering):

Draw ERD → Generate DDL.

Design first, then create.

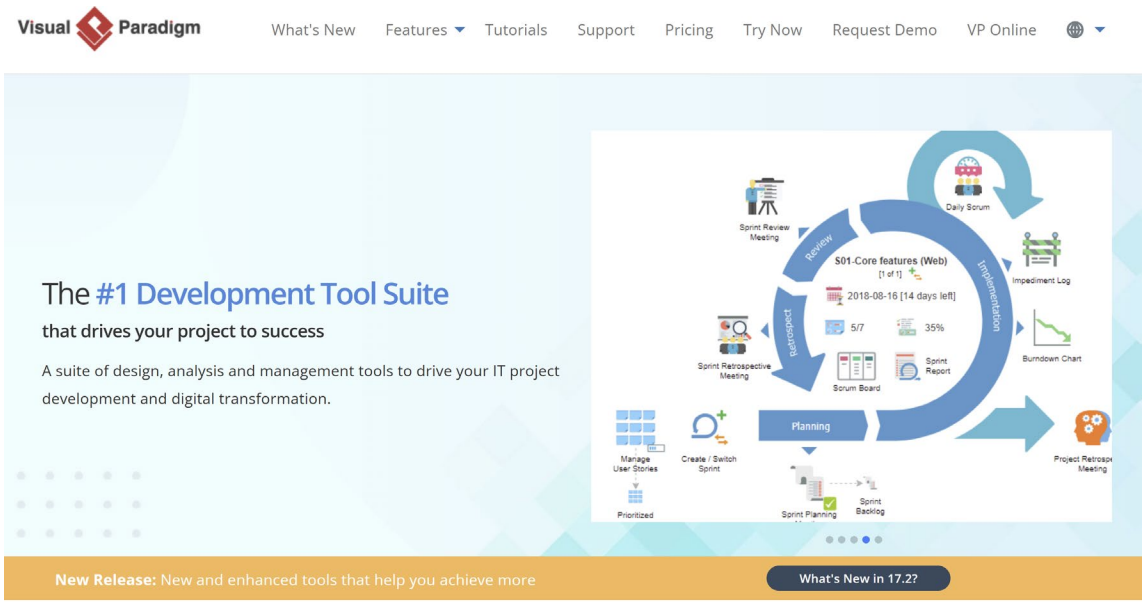
PGADMIN (Reverse engineering):

Connect to DB → Generate ERD.

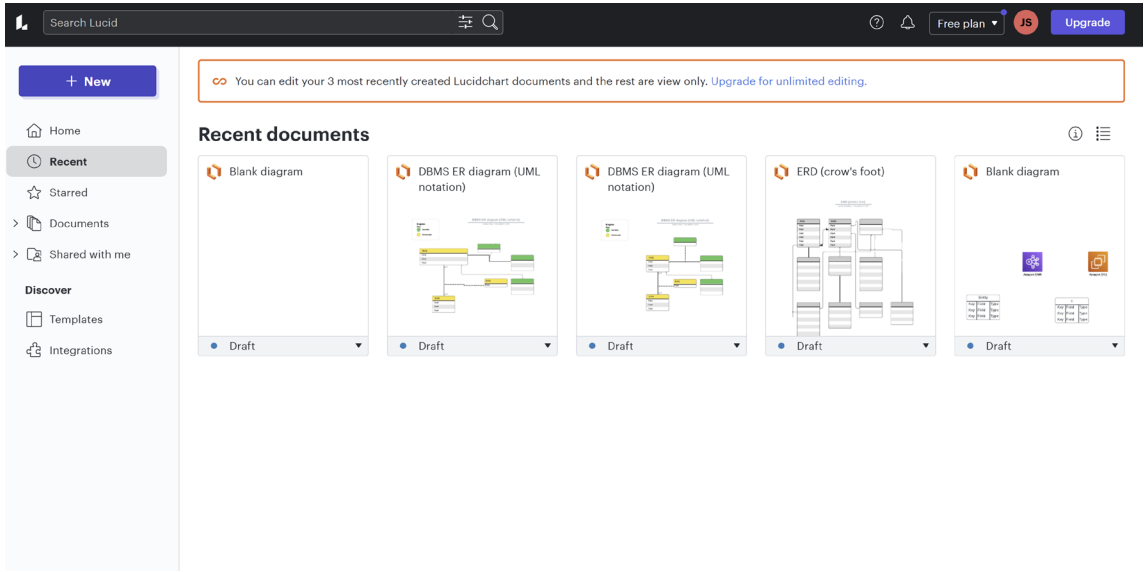
Document existing systems.

Both directions are essential skills.

Visual Paradigm



Lucidchart


Search Lucid
Free plan
JS
Upgrade
+ New
Home
Recent
Starred
Documents
Shared with me
Discover
Templates
Integrations
You can edit your 3 most recently created Lucidchart documents and the rest are view only. Upgrade for unlimited editing.
Recent documents
Blank diagram
DBMS ER diagram (UML notation)
DBMS ER diagram (UML notation)
ERD (crow's foot)
Blank diagram
Draft
Draft
Draft
Draft
Draft

NORMALIZATION THEORY

The Formal "Why" Behind Relational Design

Last Week and This Week

- WEEK 2: We saw the *symptoms*
  - Update anomaly
  - Insert anomaly
  - Delete anomaly

- WEEK 3: We learn the *cure*
  - **Normalization:** Formal rules that prevent anomalies

- Developed by Edgar Codd (IBM, 1970)
- Mathematical foundation of relational databases

THE GOALS OF NORMALIZATION

- Eliminate redundancy:
  - Each fact stored once
- Ensure integrity:
  - Changes happen in one place
- Create flexibility:
  - Easier to modify structure
- Improve maintainability:
  - Clearer organization

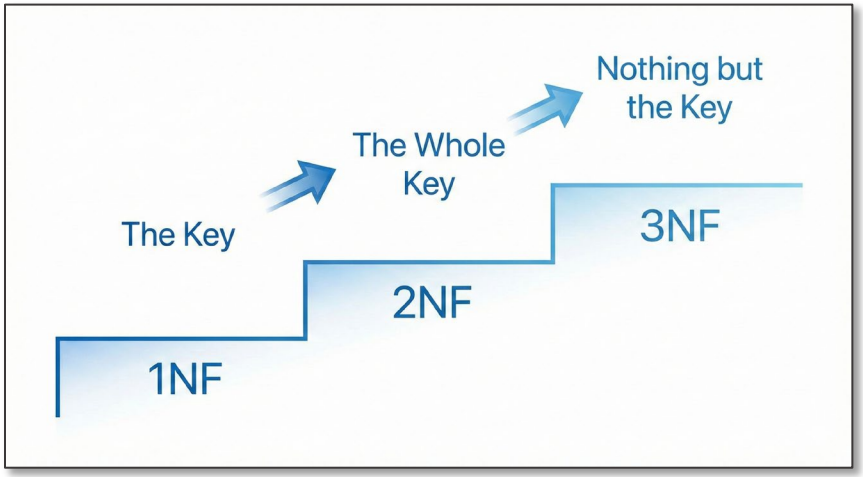
The Trade-off:

Normalized:
Consistent,
but
requires JOINs

Denormalized:
Fast reads,
but
risks inconsistency

PROGRESSIVE LEVELS OF NORMALIZATION

- **1NF** (First Normal Form):
  - Atomic values, no repeating groups
- **2NF** (Second Normal Form):
  - No partial dependencies
- **3NF** (Third Normal Form):
  - No transitive dependencies
- **BCNF** (Boyce-Codd NF):
  - Stricter 3NF



Most production OLTP systems aim for 3NF
BCNF is stricter and situational

Primary Key

DEFINITION

A column (or set of columns) that **uniquely identifies each row** in a table.

Characteristics

Unique — No duplicate values allowed

Not Null — Cannot contain NULL values

One per table — Each table has exactly one

Cat Café Example: Cats Table

CatID	Name	Breed	BirthDate
1	Whiskers	Maine Coon	2021-03-15
2	Mittens	Siamese	2020-08-22
3	Shadow	Bombay	2022-01-10
4	Luna	Persian	2019-11-05

CatID uniquely identifies each cat. No two cats share the same ID.

Foreign Key

DEFINITION

A column that creates a **link between two tables** by referencing the Primary Key of another table.

Characteristics

Referential Integrity — Values must exist in referenced table

Can be NULL — Unlike primary keys

Connects tables — Foundation of relational databases

Cat Café Example

Customers (Parent)

CustomerID	Name	Email
101	Alice Chen	alice@email.com
102	Bob Smith	bob@email.com
103	Carol Davis	carol@email.com

Reservations (Child)

ResID	CustomerID	CatID	Date
1	101	2	2024-01-15
2	102	1	2024-01-16
3	101	3	2024-01-17

CustomerID in Reservations references **CustomerID** in Customers.

FIRST NORMAL FORM

Rule:
Every cell contains ONE atomic (indivisible) value.

No repeating groups or arrays in a single cell.

Question to ask:
“Could I split this cell into multiple values?”

```
If
    yes
Then
    violates 1NF
```

THIS TABLE VIOLATES 1NF

VisitID	CustomerName	Phone	CatsVisited
V001	Maria Chen	202-555-0147	Whiskers, Mittens, Shadow
V002	James Wilson	703-555-0892	Mittens
V003	Maria Chen	202-555-0147	Shadow, Boots, Whiskers, Luna

CONVERT TO 1NF: ONE VALUE PER CELL

VisitID	CustomerName	Phone	CatName
V001	Maria Chen	202-555-0147	Whiskers
V001	Maria Chen	202-555-0147	Mittens
V001	Maria Chen	202-555-0147	Shadow
V002	James Wilson	703-555-0892	Mittens
V003	Maria Chen	202-555-0147	Shadow
V003	Maria Chen	202-555-0147	Boots
V003	Maria Chen	202-555-0147	Whiskers
V003	Maria Chen	202-555-0147	Luna

SECOND NORMAL FORM

Prerequisite: Must be in 1NF

Rule:
No partial dependencies.
Every non-key attribute depends on the ENTIRE primary key.

Only relevant when you have a COMPOSITE primary key.

Ask: "Does this attribute depend on ALL key columns, or just some?"

Composite Key

DEFINITION

A primary key consisting of **multiple columns** that together uniquely identify a row.

Why Use Composite Keys?

When **no single column** can uniquely identify rows

Common in **junction tables** for many-to-many relationships

Think of it as: Column A + Column B = unique identifier

Cat Café Example: CatVisits Junction Table

Tracks which cats interacted with customers during each visit

VisitID	CatID	InteractionType	Duration (min)
1001	1	Petting	15
1001	3	Playing	20
1002	1	Feeding	10
1002	2	Petting	25

VisitID + CatID = PRIMARY KEY

Neither VisitID nor CatID alone is unique—a visit can have multiple cats.

WHAT 2NF IS ACTUALLY FIXING

Problem Pattern:

- Composite key: (A, B)
- Attribute depends only on A or only on B

What This Means:

- Multiple facts are being stored in one table
- Attributes are attached to the wrong entity

Typical Fix:

- Split table so each fact lives with the key it depends on

THIS TABLE VIOLATES 2NF

VisitID	CatName	CustomerName	Phone	CatBreed	CatAge
V001	Whiskers	Maria Chen	202-555-0147	Persian	3
V001	Mittens	Maria Chen	202-555-0147	Tabby	5
V002	Mittens	James Wilson	703-555-0892	Tabby	5

THIS TABLE VIOLATES 2NF

Composite Key:
VisitID, CatName

VisitID	CatName	CustomerName	Phone	CatBreed	CatAge
V001	Whiskers	Maria Chen	202-555-0147	Persian	3
V001	Mittens	Maria Chen	202-555-0147	Tabby	5
V002	Mittens	James Wilson	703-555-0892	Tabby	5

SPLIT INTO TWO DIMENSION TABLES

Visits

VisitID	CustomerName	Phone	VisitDate
V001	Maria Chen	202-555-0147	2024-03-15
V002	James Wilson	703-555-0892	2024-03-16

Cats

CatName	CatBreed	CatAge
Whiskers	Persian	3
Mittens	Tabby	5
Shadow	Black Shorthair	2

JOIN DIMENSION TABLES WITH A FACT TABLE

Visits - Cats

VisitID	CatName	VisitID
V001	Whiskers	V001
V001	Mittens	V001
V002	Mittens	V002

THIRD NORMAL FORM

Prerequisite: Must be in 2NF

Rule:

- No transitive dependencies.
- Non-key attributes cannot depend on OTHER non-key attributes.

Ask:

"Does this non-key column determine another non-key column?"

THIRD NORMAL FORM

Problem Pattern:

- Key → A
- A → B
- Therefore: Key → B *(indirectly)*

Why This Is Bad:

- Same fact repeated across many rows
- Updates must be made in multiple places

Typical Fix:

- Extract the non-key → non-key relationship into its own table

THIS TABLE VIOLATES 3NF

VisitID	CustomerName	Phone	ZipCode	City
V001	Maria Chen	202-555-0147	20001	Washington
V002	James Wilson	703-555-0892	22201	Arlington

Dependency chain:

- VisitID → ZipCode
- **ZipCode → City**

Fixing 2NF does NOT guarantee 3NF.

Each normal form eliminates a *different dependency pattern*.

AGAIN, SPLIT INTO TABLES

Visits

VisitID	CustomerName	Phone	ZipCode
V001	Maria Chen	202-555-0147	20001
V002	James Wilson	703-555-0892	22201

Zip_Codes

ZipCode	City
20001	Washington
22201	Arlington

Candidate Key

DEFINITION

Any column (or combination) that **could uniquely identify** a record. A table may have multiple candidate keys.

Key Insight

One candidate key is **chosen as the primary key**. The others remain as **unique constraints**.

Characteristics

Minimal — No unnecessary columns

Unique — Guarantees uniqueness

Cat Café Example: Customers Table

CustomerID	Name	Email	Phone
101	Alice Chen	alice@email.com	555-0101
102	Bob Smith	bob@email.com	555-0102
103	Carol Davis	carol@email.com	555-0103

PRIMARY KEY
CustomerID

CANDIDATE KEY
Email

CANDIDATE KEY
Phone

All three could uniquely identify a customer, but CustomerID was chosen.

BOYCE–CODD NORMAL FORM (BCNF)

Problem Pattern:

- For every functional dependency **X → Y**
- **X must be a candidate key**

Stricter than 3NF

ShiftID	Manager	ManagerPhone
S001	Alex	202-555-0101
S002	Bri	202-555-0199
S003	Alex	202-555-0101

COMPARING THE NORMAL FORMS

Normal Form	Eliminates	Key Idea
1NF	Repeating groups	Atomic structure
2NF	Partial dependency	Entire composite key
3NF	Transitive dependency	No non-key → non-key
BCNF	Non-key determinants	Determinants must be keys

DATABASE ORGANIZATION

Schemas, Databases, and Governance

Why Database Organization Matters

- Prevents silent analytical errors
- Enables governance and compliance
- Supports team ownership and accountability
- Reduces security and policy risk

THE CAT CAFÉ:
SAME DATA, DIFFERENT QUESTIONS

Operational (OLTP)

- Did a customer successfully place an order?
- Which cats are currently available?
- Update inventory after a sale

Analytical (OLAP)

- Which drinks sell best on weekends?
- How does cat adoption correlate with visit time?
- Monthly revenue trends by location

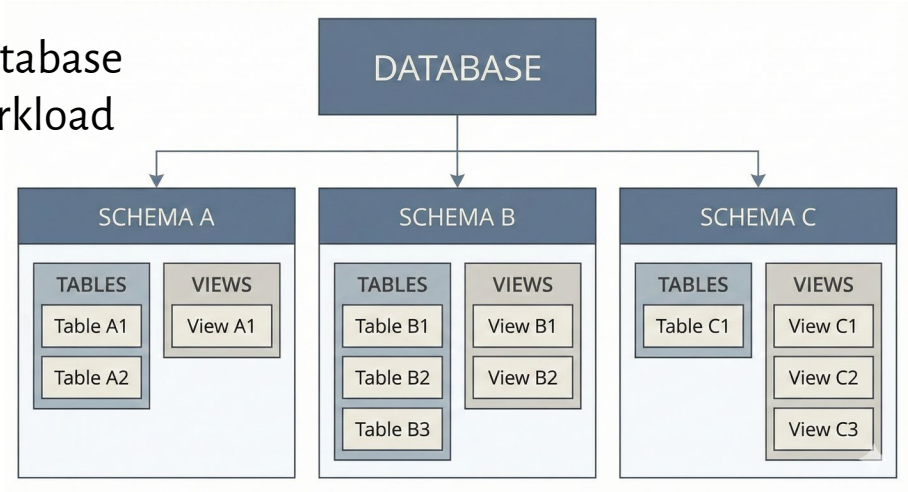
SCHEMAS: NAME CONTAINERS

Schemas: Named Containers

- A schema is a named container inside a database
- Groups related objects by purpose and workload

Examples

- operations.orders
- operations.cats
- analytics.fact_sales
- analytics.dim_cat



Hierarchy

Database

→ Schema

→ Tables / Views / Functions

SCHEMA USE CASES

- Namespacing:
  - Avoid naming collisions
  - operations.customers vs analytics.customers
- Logical organization:
  - Group by domain
  - OLTP vs OLAP workloads
- Security boundaries:
  - Grant permissions at schema level
- Team ownership:
  - App team vs analytics team

TWO WORLDS OF SCHEMA DESIGN

OLTP (Operations Schema)

- Write-intensive
- Normalized (3NF)
- Current state
- Example: operations.orders, operations.customers

OLAP (Analytics Schema)

- Read-intensive
- Intentionally denormalized
- Historical data
- Example: analytics.fact_sales, analytics.dim_customer

INTENTIONAL DENORMALIZATION

Operations (OLTP)

- Normalize to prevent anomalies
- Example:
  - customers → addresses → cities

Analytics (OLAP)

- Denormalize to reduce JOINs
- Example:
  - dim_customer includes name, city, region

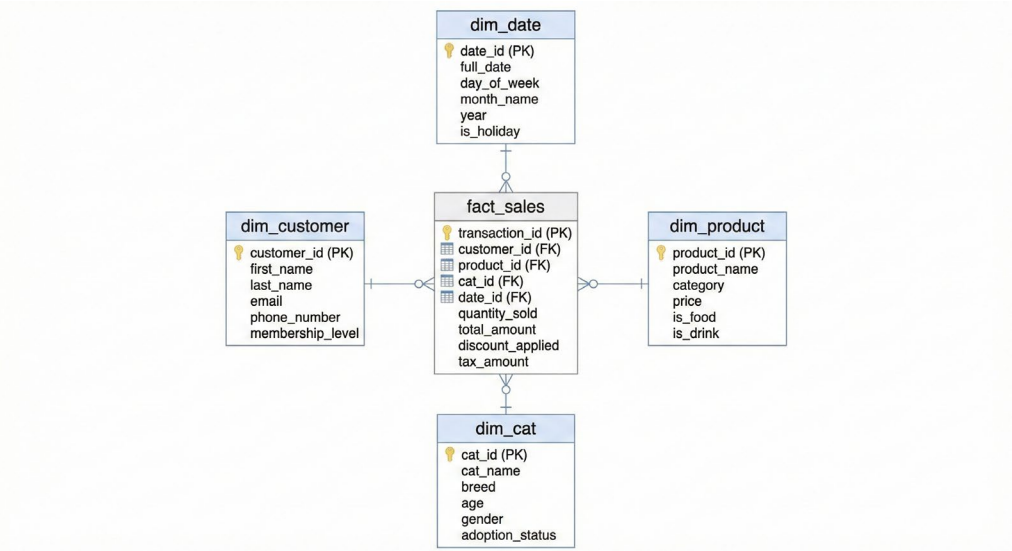
STAR SCHEMA: THE OLAP WORKHORSE

fact_sales

- visit_id, sale_amount, items_sold
- Foreign keys to all dimensions

Dimensions

- Customers,
- products,
- cats,
- dates
- Denormalized attributes



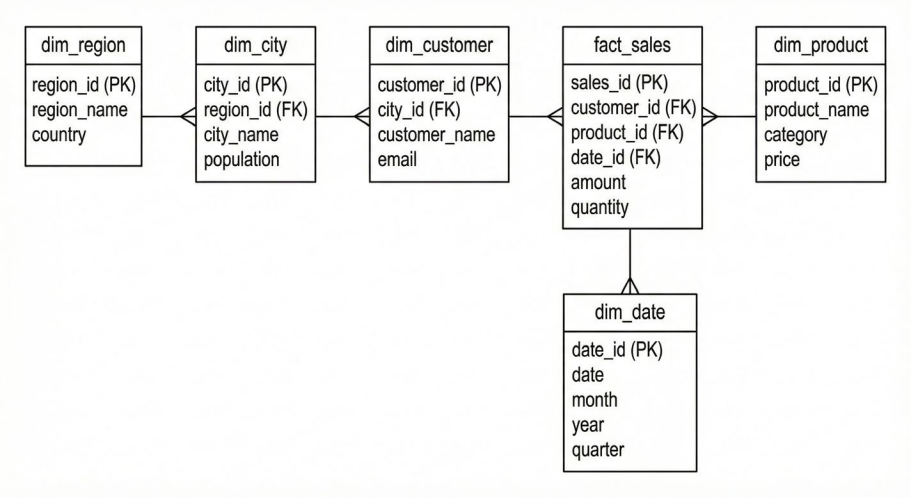
SNOWFLAKE SCHEMA: NORMALIZED DIMENSIONS

Why use it?

- Large, frequently changing dimensions
- Cleaner updates

Why avoid it?

- More JOINs
- Slower analytics



STAR vs SNOWFLAKE: DECISION GUIDE

Factor	Star	Snowflake
Query speed	Faster	Slower
Storage	More	Less
Maintenance	Harder	Easier
BI tools	Best support	Mixed

Start with star schema unless you have a strong reason not to.

OLAP CUBES: MULTI-DIMENSIONAL THINKING

- Dimensions: time, product, cat
- Measures: revenue, visits
- Enables slicing and aggregation

SCHEMAS ARE NOT "DISCONNECTED TABLES"

- ❌ *Wrong:*
"Operations and analytics schemas can't interact"

- ✅ *Right:*
Cross-schema access is allowed by design

Example:
ETL reads from operations.orders
Writes to analytics.fact_sales

```
SELECT *
FROM operations.orders o
JOIN sales.customers c
  ON o.customer_id = c.id;
```

SCHEMAS vs DATABASES

Use Schemas

- OLTP + OLAP in same platform
- Shared governance
- Easy cross-domain joins

Use Multiple Databases

- Strict legal isolation (PII)
- Independent lifecycle management

MENTAL MODELS FOR DATA ORGANIZATION

Concept	Analogy	Cat Café Meaning
Database	Building	Entire café system
Schema	Floor	Operations vs analytics
Table	Room	Orders, cats, sales
View	Window	Analyst-safe access
Permission	Lock	Who can see what

INDEXES

Optimizing for Read vs. Write

What Is an Index?

Database Indexes: Fast Lookups at Scale

An index is a separate data structure that maps:

`value → location of rows`

Think of it like:

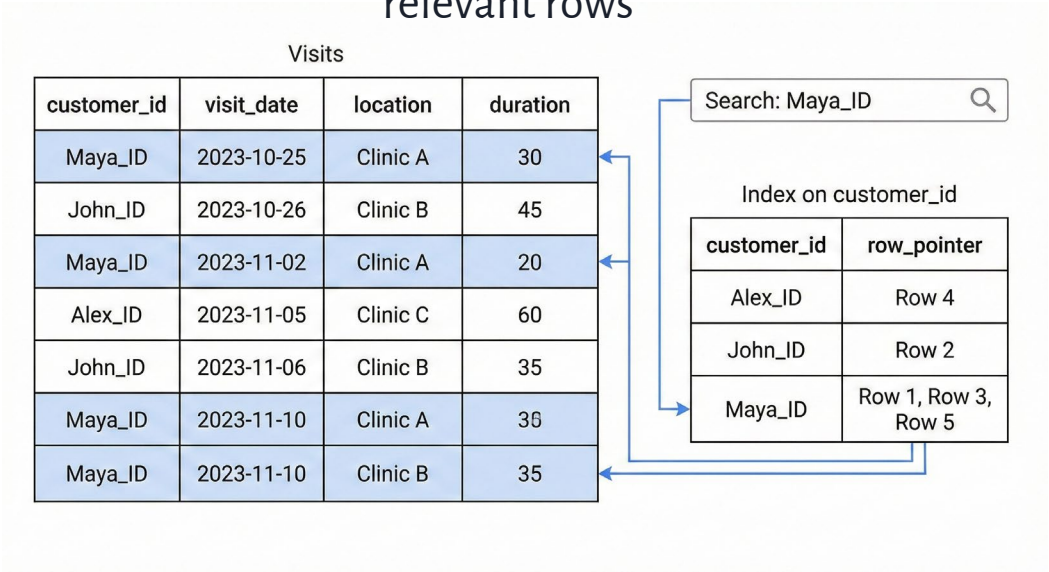
- A book index
- A library card catalog

Without an index:

- Database performs a full table scan

With an index:

- Database can jump directly to relevant rows



Why Indexes Matter in Massive Data Systems

Index benefits grow nonlinearly with data size
Difference between:

- 10,000 rows → manageable scan
- 100 million rows → unacceptable latency

Indexes reduce:

- I/O
- CPU
- Network movement (distributed systems)

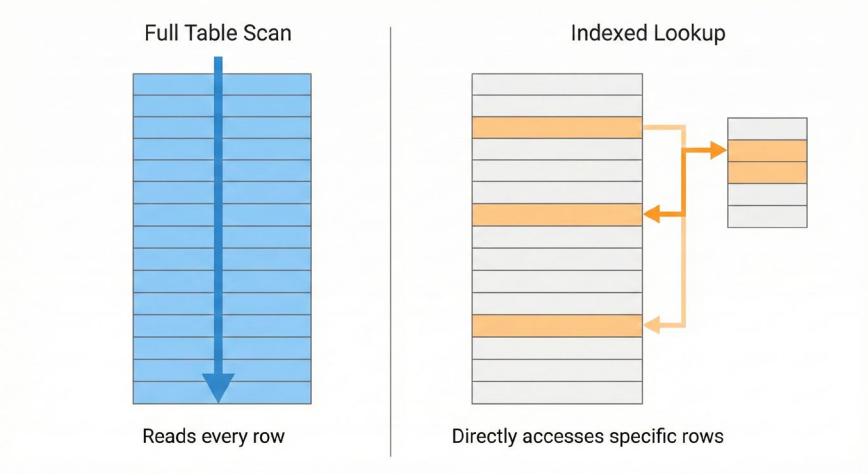
Table Size	No Index	With Index
10K rows	Fast enough	Slightly faster
1M rows	Noticeable delay	Fast
100M rows	Very slow	Acceptable
1B rows	Often unusable	Still usable

Indexes and Filtering (WHERE Clauses)

Indexes help most when used in WHERE conditions that significantly reduce rows.

```
SELECT *
FROM visits
WHERE customer_id = 42;
```

- Indexed customer_id:
  - fast lookup
- No index
  - scan entire visits table

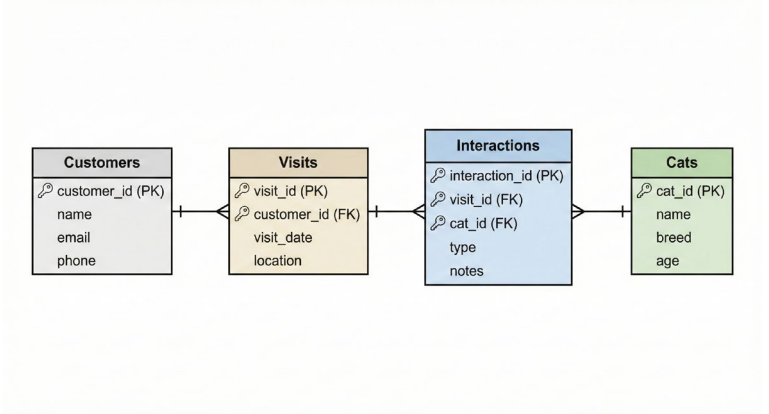


Indexes and JOIN Performance

Indexes on join keys dramatically reduce join cost.

```
customers.id → visits.customer_id
visits.visit_id → interactions.visit_id
interactions.cat_id → cats.cat_id
```

- Without indexes:
  - Nested loops
  - Exploding comparisons
- With indexes:
  - Efficient lookups
  - Scales to millions of rows



Join Column	Index Needed?	Why
Primary key	Usually yes	Fast lookup
Foreign key	Yes	Join efficiency
Text fields	Rarely	High cost
Low-cardinality flags	No	Poor selectivity

Indexes Are Not Free

- Costs:
  - Extra storage
  - Slower INSERT / UPDATE / DELETE
  - Maintenance overhead
  - Poor indexes can hurt performance

Benefit	Cost
Faster reads	Slower writes
Better joins	More storage
Scalable queries	Index maintenance
Lower latency	Design complexity

When NOT to Use an Index

- Very small tables
- Columns where almost every row matches
- Columns used only in SELECT (not WHERE or JOIN)
- Temporary or staging tables

Indexes are for questions, not storage.

SQL JOINs

Recombining Normalized Data

Recombining Normalized Data

JOIN = Match rows where key values align

```
ON customers.id = visits.customer_id
```

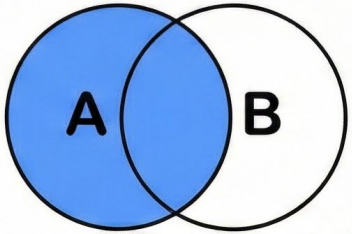
"Connect rows where the customer ID matches."

Mental Model:

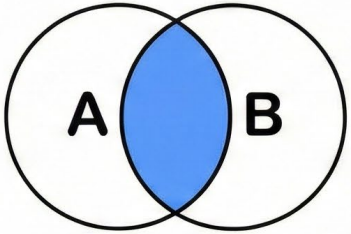
- Tables represent *entities*
- Keys represent *relationships*
- JOINs reconstruct the real-world process from normalized pieces

CHEATSHEET

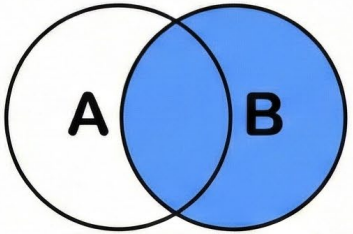
SQL JOINS



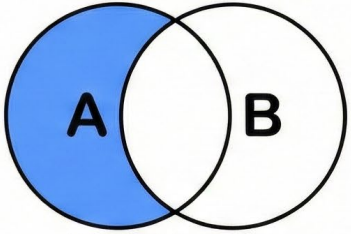

```
SELECT A.id, A.name, B.value
FROM tableA AS A
LEFT JOIN tableB AS B
ON A.key = B.key;
```



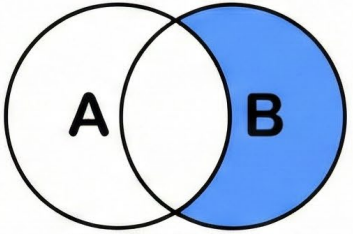

```
SELECT A.id, A.name, B.value
FROM tableA AS A
INNER JOIN tableB AS B
ON A.key = B.key;
```



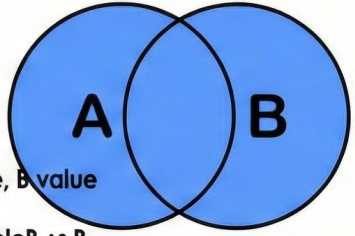

```
SELECT A.id, A.name, B.value
FROM tableA AS A
RIGHT JOIN tableB AS B
ON A.key = B.key;
```



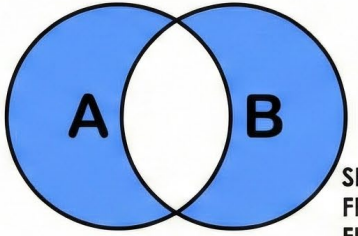

```
SELECT A.id, A.name
FROM tableA AS A
LEFT JOIN tableB AS B
ON A.key = B.key
WHERE B.key IS NULL;
```



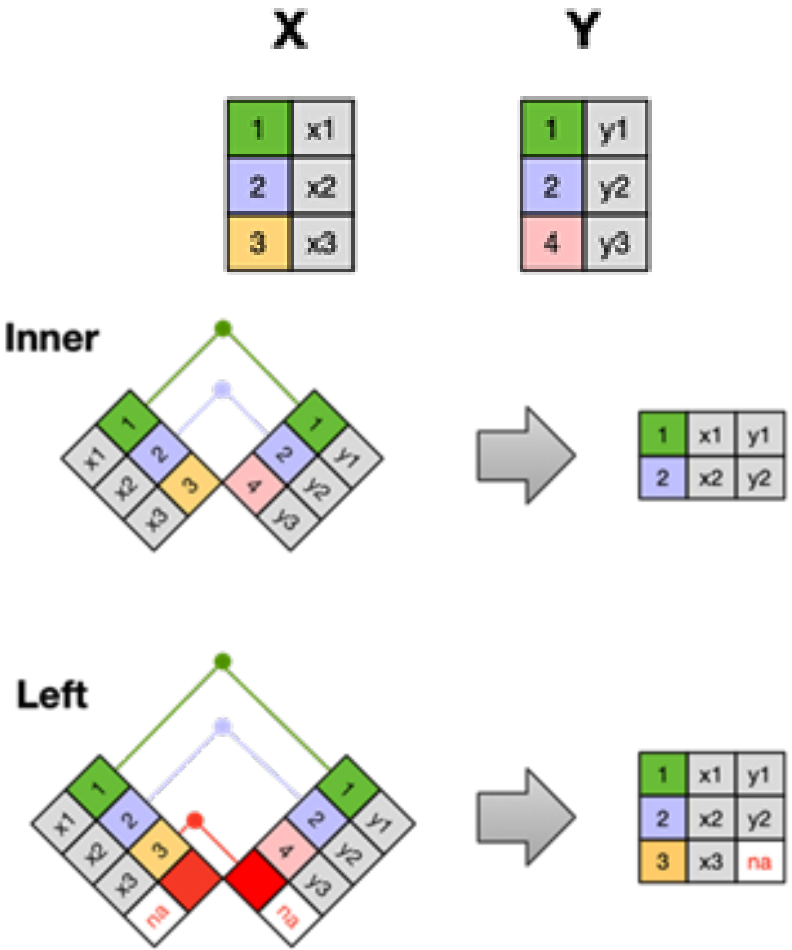

```
SELECT B.id, B.value
FROM tableA AS A
RIGHT JOIN tableB AS B
ON A.key = B.key
WHERE A.key IS NULL;
```



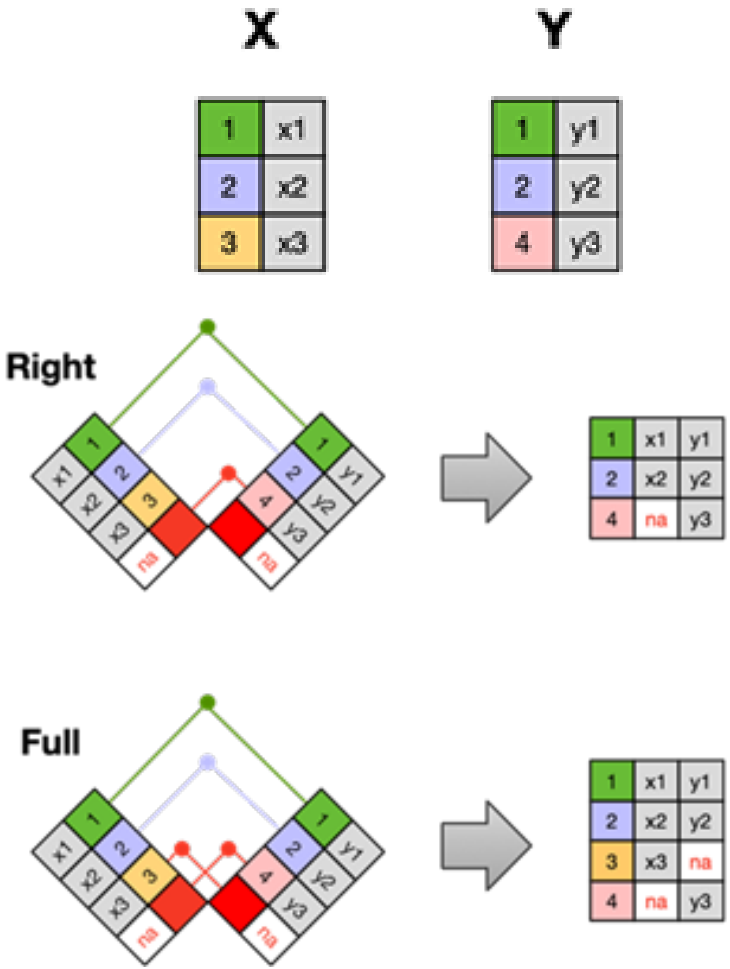

```
SELECT A.id, A.name, B.value
FROM tableA AS A
FULL OUTER JOIN tableB AS B
ON A.key = B.key;
```




```
SELECT A.id, A.name, B.id, B.value
FROM tableA AS A
FULL OUTER JOIN tableB AS B
ON A.key = B.key
WHERE A.key IS NULL
OR B.key IS NULL;
```



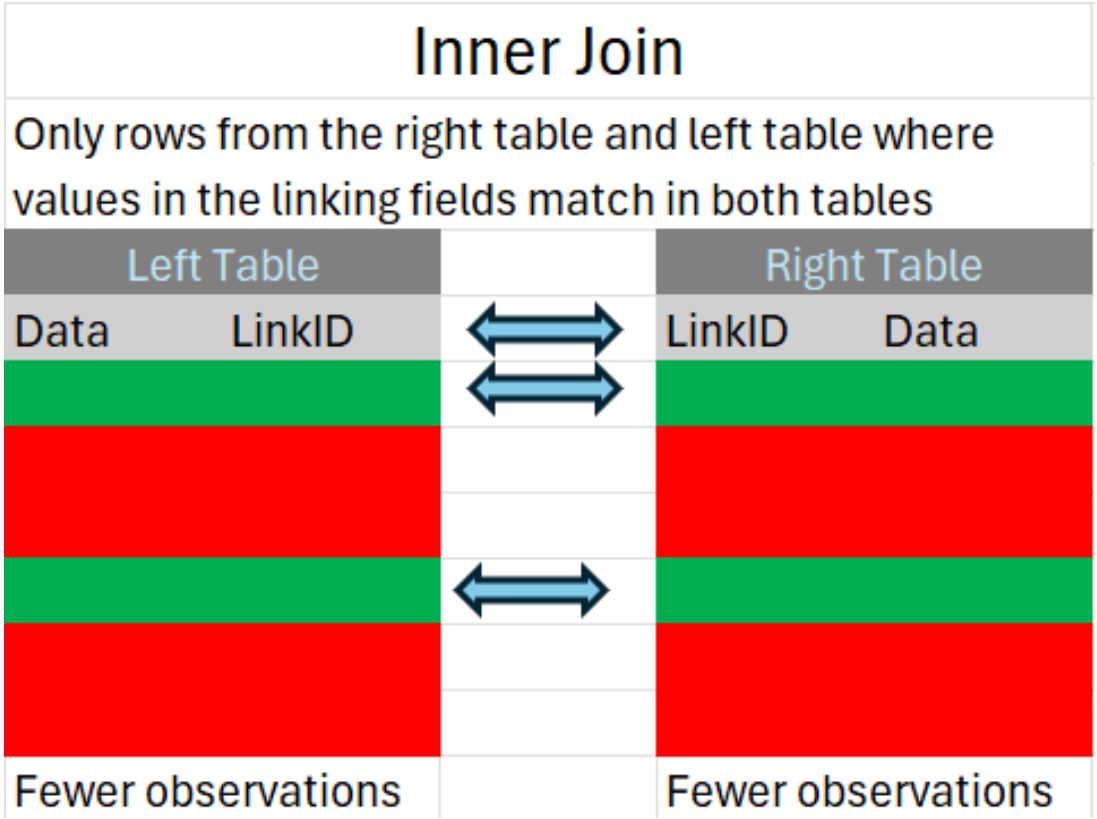
Source: [R for Data Science]



Source: [R for Data Science]

Inner Join

```
SELECT …
FROM customers c
INNER JOIN visits v
  ON c.id = v.customer_id
```



INNER JOIN: ONLY MATCHING ROWS

What It Returns

- Rows only where the relationship exists
- Records without matches are excluded

Plain-English Rule

- “Only keep customers who actually had visits.”

When to Use INNER JOIN

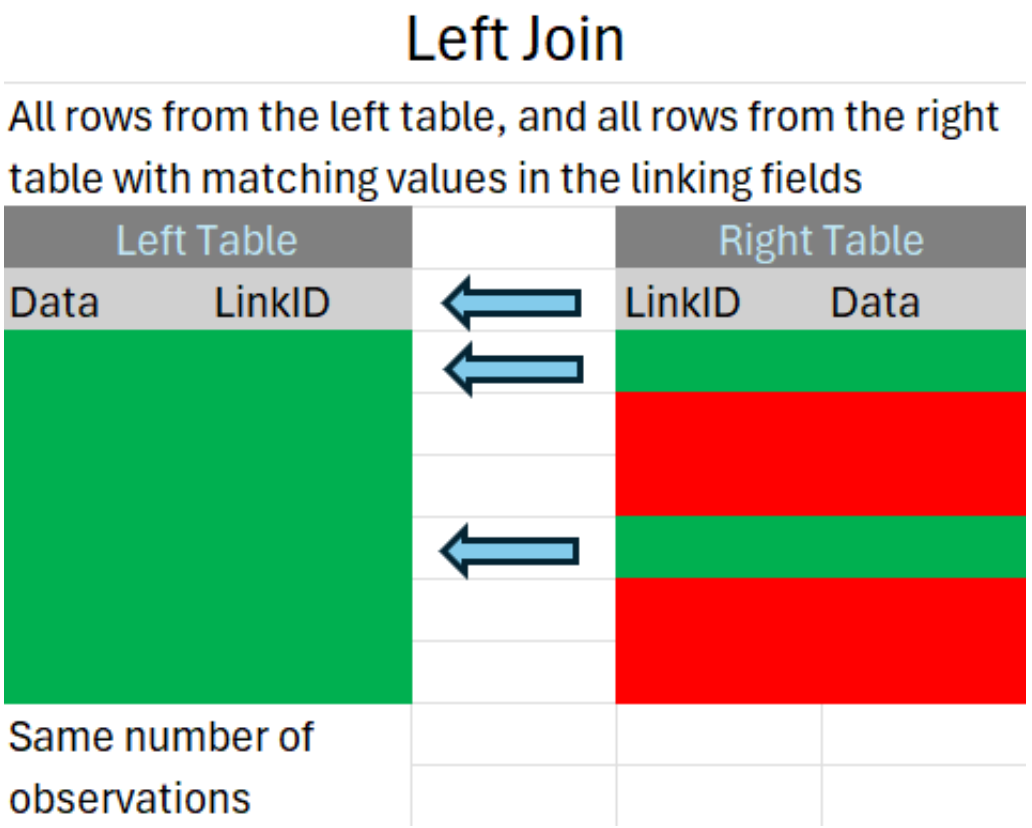
- You care only about *observed relationships*
- You are analyzing completed transactions, interactions, or events
- You want to exclude incomplete or missing data

Common Use Cases

- Sales with confirmed orders
- Users with logged activity

Left (Outer) Join

```
SELECT …
FROM customers c
LEFT JOIN visits v
  ON c.id = v.customer_id
```



LEFT JOIN: All Left Rows + Matches

What It Returns

- Every row from the left table
- Matching rows from the right
- NULL where no match exists

Plain-English Rule

- “Keep all customers — show visits where they exist.”

When to Use LEFT JOIN

- You want a complete population
- Missing relationships are analytically meaningful
- You want to detect non-participation

Typical Questions This Answers

- Who *did not* engage?
- Which entities lack downstream activity?

FULL OUTER JOIN: All Rows from Both Tables

What It Returns

- Matched rows
- Left-only rows
- Right-only rows

When to Use FULL OUTER JOIN

- Auditing data pipelines
- Identifying orphaned records
- Reconciling two systems of record

Example Use Case

- “Which customers exist in CRM but not billing, and vice versa?”

Diagram Cue

- Entire Venn diagram shaded

CHOOSING THE CORRECT JOIN

Question you're asking	JOIN to use
Only matched records?	INNER
All primary entities, even if missing data?	LEFT
Audit mismatches across systems?	FULL OUTER
Want completeness over strict accuracy?	LEFT
Want strict relational integrity?	INNER

CHAINING JOINS: FOLLOW THE ERD PATH

Key Rule

JOINs should follow the real-world process path, not convenience.

Example Question

“Which cats did Maya interact with?”

Customer → Visit → Interaction → Cat

```
FROM customers c
JOIN visits v
  ON c.id = v.customer_id
JOIN interactions i
  ON v.visit_id = i.visit_id
JOIN cats cat
  ON i.cat_id = cat.cat_id
```

COMMON JOIN MISTAKES

1. Missing ON Clause
Result: Cartesian product (A × B)
Symptom: Row counts explode

2. Wrong Join Keys
Result: Plausible-looking but incorrect data
Fix: Always join **foreign key → primary key**

3. One-to-Many Multiplication
Result: Duplicate rows
Reality: The database is correct, BUT your expectation isn't

4:

```
NULL = NULL       -- FALSE
NULL IS NULL      -- TRUE
```

NULL means "unknown," not "empty."

BEYOND RELATIONAL TABLES

JSON, Parquet, and Modern Data Formats

WHEN TABLES AREN'T ENOUGH

Structured (Relational): Defined schema. Good for OLTP.
Semi-Structured (JSON): Self-describing. Flexible.
Unstructured (Images/Text): Schemaless. Most scalable.

Relational: Strength = strict schema, ACID | Limit = rigid schema, overhead

JSON: FLEXIBLE, NESTED STRUCTURE

Pros: Flexible schema, natural for APIs, handles nesting
Cons: No referential integrity

PostgreSQL supports JSONB (JSON with database powers)

PARQUET: OPTIMIZED FOR ANALYTICS

Row-oriented: [id, name, date]
Column-oriented (Parquet): [all ids], [all names]

Benefit: Query only columns you need. Better compression.
Use case: Big Data (Spark, AWS, Databricks)

FORMAT DECISION GUIDE

- Transactions → Relational (PostgreSQL)
- APIs/Flexible → JSON
- Large Analytics → Parquet

No single format is best for everything.

TAKEAWAYS

Key Concepts and Next Steps

AVOID THESE MISTAKES

❌ "Schemas are disconnected silos" → They're namespaces
❌ "More normal forms = always better" → Trade-offs exist (OLTP vs OLAP)
❌ "LEFT JOIN excludes non-matches" → It includes them as NULL

When confused, ask: What's the use case?

WHAT WE COVERED TODAY

ERDs: Draw before you code
Normalization: The "why" behind the design
Organization: Schemas and databases structure the world
Indexes: Trade write speed for read speed
JOINs: Follow your ERD
CTEs: Make queries readable

COMING UP

Next Week: Cloud Fundamentals (AWS)

Today's Lab: SQLite backend, Visual Paradigm, Murder Mystery CTEs

Deliverable: SQL Company Analysis (Due before Week 4)